

XDR Security Engagement Report

Purple-Team Stress Test of the Big Chemistry Threat-Monitoring System

Big Chemistry Security Operations

June 10, 2026

1. Executive Summary

We ran a full, realistic attack simulation against the production threat-monitoring platform (UATMS / XDR) to answer one question: **if a real attacker came at this system, would we see them — and could we stop them?**

We attacked from two directions (from the internet, and from inside a compromised container) at three attacker skill levels (script-kiddie, experienced hacker, and nation-state). For every attack we watched the security dashboards to record whether the system **detected**, **blocked**, or **missed** it.

Bottom line. The monitoring pipeline is genuinely alive and catching attacks — but the stress test exposed **one critical and three high-severity gaps that a capable attacker could use to take over the AWS account or quietly steal research data**. Two real problems were found *and fixed live during the test*. The remaining red items **must be fixed before this is considered production-ready**.

Severity	Count	Examples
Critical	1	Any GitHub branch/PR can become full AWS administrator
High	3	Undetectable network C2/exfil; cluster-persistence invisible to the SIEM
Medium	3	Alert noise, DNS-exfil blind spot, open internal lateral movement
Fixed during test	2	“Kill attacker tools” was broken (now works); SIEM agents were offline (now restored)
Passed (working well)	5+	Container-escape blocked; pod→cloud-credential theft blocked; key software patched

2. What We Did (in plain terms)

A “purple team” exercise means the attacker (us) and the defender (us, watching the dashboards) work together so every attack is measured against the monitoring system.

We ran **three attack campaigns**:

1. **“Pipeline Poison” — break in from the internet.** Target the public-facing CI/CD build server and the cloud account.
2. **“Kernel Break” — escape a hacked container.** Assume an attacker already has a foothold in a pod (e.g. via phishing or a vulnerable app) and try to break out to the underlying server and the cloud.
3. **“Token Heist” — steal data & stay hidden.** Try to pivot to AWS secrets, smuggle data out, and install hidden persistence.

Each was run at three skill levels so we could see not just *“does the alarm work”* but *“does it still work against someone who’s trying to be quiet.”*

3. Findings at a Glance

#	Severity	Finding (plain language)	Business impact	Status
F-09	CRITICAL	Any branch or pull request in the GitHub repo can assume a full-admin AWS role	Total account takeover — delete/steal all research data, shut down the platform	OPEN — fix first
F-08	HIGH	Network sensors (Suricata/Zeek) cannot see attacker command-and-control or data theft (traffic is encrypted before they see it)	A patient attacker exfiltrates research IP silently	OPEN
F-10	HIGH	Attacker actions against the cluster (hidden scheduled jobs, permission changes) never reach the SIEM	Persistence & tampering go unnoticed	OPEN
F-05	MEDIUM	DNS-based data smuggling from a pod is invisible to the network monitor	Covert exfiltration channel	OPEN
F-02	MEDIUM	Detection works but floods low-value alerts, hiding real ones	Alert fatigue → missed incidents	OPEN
F-07	MEDIUM	A single hacked pod can freely reach the database / login / search services	Faster lateral spread after a breach	OPEN (review)
F-01	HIGH → FIXED	The “automatically kill hacker tools” rule did nothing (a logic bug)	Attacker tools ran freely	FIXED & re-tested live
F-03	HIGH → FIXED	All SIEM sensor agents had silently disconnected — the dashboard was blind	No telemetry at all	FIXED live + hardened
F-06	PASS	A hacked pod could not steal the server’s cloud credentials or reach the cluster API	Escape-to-cloud blocked	Working
—	PASS	The container runtime (runc) is patched against the latest public escape exploits	No container breakout	Working

4. The Critical & High Issues — what they mean for the business

F-09 (CRITICAL) — A pull request can become AWS root

The automation that lets GitHub deploy to AWS trusts **any** activity in the repository (`...:*`) and grants it **AdministratorAccess**. There is no restriction to the `main` branch or an approved environment.

Attack scenario: a Fontys student opens a pull request, or an attacker phishes one contributor’s GitHub account, and adds a few lines to a workflow file. On the next run they receive **full administrator credentials to the entire AWS account** — and because this rides the *normal, expected* login path, the cloud audit logs see it as a routine deployment. They could delete every backup, exfiltrate all of Big Chemistry’s research data, or hold the platform to ransom.

Fix: restrict the trust to the `main` branch / a protected “production” environment with required reviewers, and replace `AdministratorAccess` with a minimal deploy-only permission set.

F-08 (HIGH) — Network C2 and data theft are invisible

This is the most important *detection* gap. The internal network sensors (Suricata, Zeek) sniff the physical network card — but Cilium **WireGuard encrypts pod-to-pod traffic before it reaches that card**. We confirmed this live: every flow from our attacker pod showed up only as encrypted volume/timing data, never readable content. On top of that, Suricata is in *detect-only* mode and has no signature for novel C2.

Attack scenario: an attacker who lands inside a container (phishing, a vulnerable web app) opens an encrypted command-and-control channel to the internet and slowly streams research data out. The network sensors see noise; no alarm fires. The **only** thing that caught our activity was Tetragon noticing the *tool* (`dig/curl`) being launched — so an attacker who beacons from an already-running process, or a custom program, would be **completely undetected**.

Fix: detection alone can’t win here — **prevention** must carry it: lock down container egress with an allow-list (so C2 to an unknown destination is *dropped*, as the cloud-metadata and cluster-API endpoints already are), and add Layer-7 visibility at a decryption point (Hubble L7 / Cilium Envoy).

F-10 (HIGH) — Cluster persistence never reaches the SIEM

EKS audit logging is switched on, but the logs stop at AWS CloudWatch and are **not forwarded to Wazuh**. We planted a hidden scheduled job (a classic persistence trick) — **zero alerts**.

Attack scenario: after a breach, the attacker installs a scheduled job that re-opens their access every 5 minutes, or quietly grants themselves more permissions. None of it appears on the security dashboard the team watches.

Fix: forward the EKS audit log group into Wazuh (or enable GuardDuty EKS Protection) and switch on the detection rules that already exist but aren’t wired up.

F-05 (MEDIUM) — DNS smuggling blind spot

A related exfiltration path: data hidden inside DNS lookups from a pod never appears in the network monitor (same encryption/routing reason as F-08). **Fix:** log DNS at the cluster DNS server and feed it to the SIEM.

5. What’s Already Working (for balance)

A monitoring system is only credible if we report the wins honestly too:

- **Container escape is blocked.** The latest public container-breakout exploits (runc, Nov 2025) are patched — we confirmed the version and did not attempt a breakout that couldn’t succeed.
- **A hacked pod cannot steal cloud credentials.** Both the server’s instance credentials and the cluster API were unreachable from our attacker pod — the two classic “escape to cloud” pivots **failed**.
- **Key software is patched.** Cilium and Wazuh are on versions that fix recent critical vulnerabilities.
- **The alarm system can’t be silenced from inside.** GuardDuty is centrally managed, so even an account-level attacker cannot disable it.

- **The detection pipeline is live and rich** — 215,000+ alerts in 24h, mapped to the MITRE ATT&CK framework (see evidence below).
-

6. Problems We Found *and Fixed* During the Test

F-01 — “Kill attacker tools” was broken; now it works

The flagship runtime control was supposed to instantly kill hacker tools (`nmap`, `nc`, etc.) the moment they run. **It did nothing** — a subtle logic bug meant the rule inspected the *calling shell* instead of the *tool being launched*, so it never matched. We confirmed `nmap` ran to completion untouched.

We corrected the rule (match the program being executed, not the caller), applied it, and **re-tested live: `nmap` and `nc` are now killed instantly** (see the SIGKILL evidence in §7). No legitimate workloads were affected.

F-03 — The SIEM had gone blind; now restored

Mid-engagement, **all four sensor agents had silently disconnected** from the SIEM (a recurring enrolment deadlock). The dashboard was receiving nothing. We diagnosed the root cause, cleared it, added a durable self-healing setting, and **restored all four agents** — telemetry is flowing again and is hardened against recurrence.

7. Evidence

The monitoring system is live and catching attacks — 215K+ alerts in 24 hours, mapped to MITRE ATT&CK, all four sensor agents reporting:

Runtime detections (Tetragon) flowing end-to-end into the SIEM — 45,202 events, showing shell-spawn and network-tool executions inside containers:

Network visibility (Hubble) — our simulated attacker pod (`pentest-netshoot`, far left) reaching the database, login, and search services (lateral movement, finding F-07):

After the fix (F-01) — Tetragon now **kills** attacker tools on sight, raising critical (level 14) alerts. This is the corrected control working:

8. Remediation Plan (prioritized)

Fix now (before any production use)

1. **F-09** — Scope the GitHub→AWS trust to `main` / protected environment; drop `AdministratorAccess` to least-privilege. (*Single biggest risk.*)
2. **F-10** — Forward EKS audit logs into Wazuh; enable the existing K8s detection rules.

Fix this sprint (close the C2/exfil blind spot)

3. **F-08 / F-05** — Enforce a container **egress allow-list** (drop traffic to unknown destinations); add DNS logging at CoreDNS → SIEM; add L7 visibility via Hubble/Envoy.
4. **F-02** — Tune the noisy `setuid` rule so real alerts aren’t buried.
5. **F-07** — Tighten per-workload network policies between app tiers.

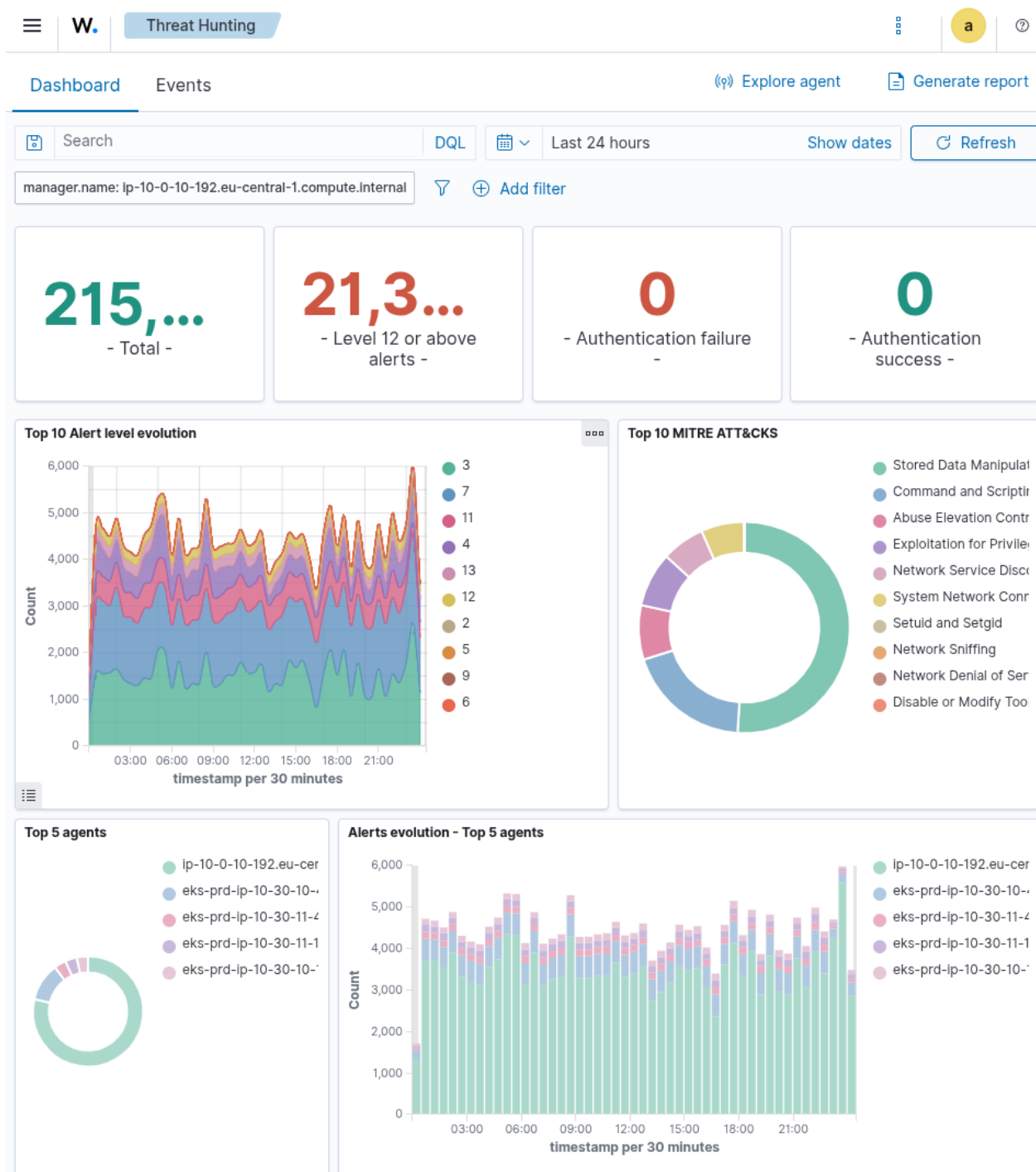


Figure 1: SIEM dashboard — alert volume, MITRE ATT&CK breakdown, and the four reporting agents

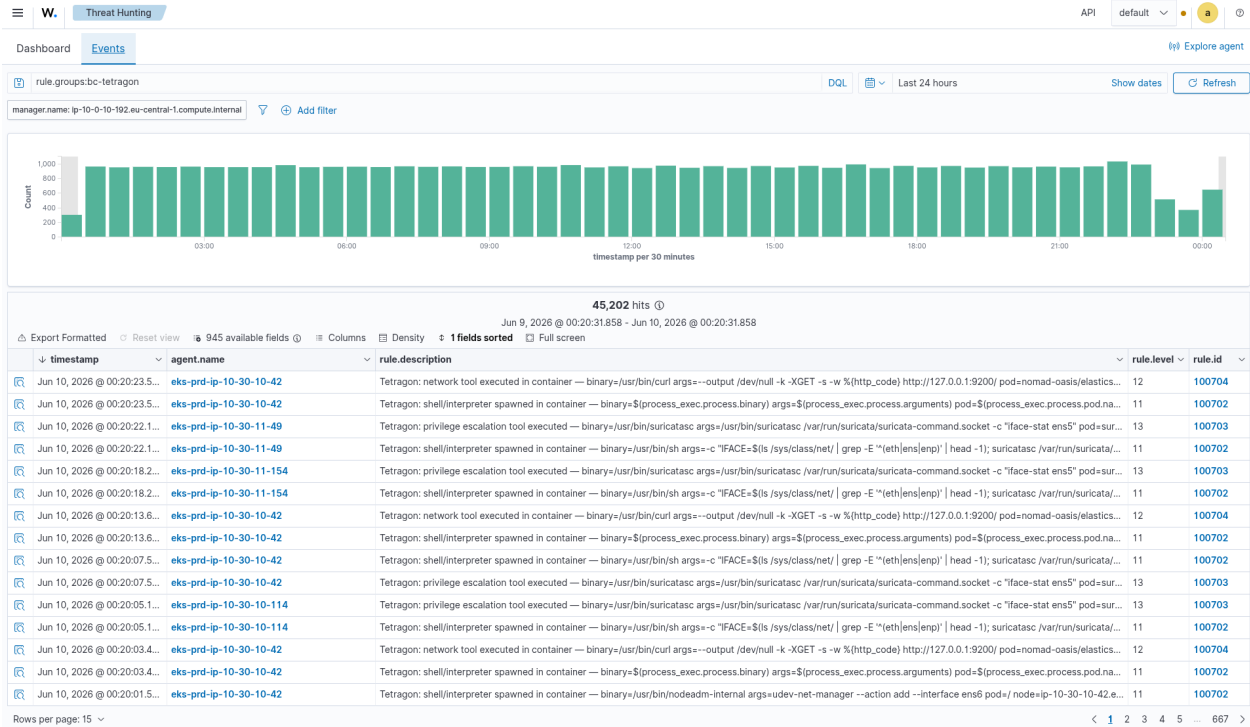


Figure 2: Tetragon runtime detections in Wazuh (45,202 events)

Already done (this engagement)

6. **F-01** — Tetragon “kill malicious tools” rule corrected and re-tested live. (*Pending: merge to the repo via review.*)
7. **F-03** — SIEM agents reconnected and hardened against recurrence.

Hardening (defense-in-depth)

8. Set the server metadata hop-limit to 1 (make the pod→cloud-credential block explicit).
9. Make the public EKS API and the public CI runner private / IP-restricted.

9. Conclusion

The XDR platform’s core thesis — **kernel-level (eBPF) monitoring sees what the network can’t** — held up under stress: Tetragon was the only sensor that caught in-container activity, exactly as designed. But the same test proved the **limits** of that design: once traffic leaves the kernel it’s encrypted and unseen, so **command-and-control** and **data exfiltration** are **currently undetectable**, and **one CI/cloud misconfiguration hands an attacker the entire account**.

These are fixable. With the egress allow-listing, audit-log forwarding, and the GitHub trust-scoping above, this system goes from “*detects the obvious*” to “*genuinely hard to breach and quietly exfiltrate from.*” Two of the gaps were already closed during the engagement; the rest are scoped and prioritized above.

Full technical detail, attack-by-attack matrix, and MITRE mappings: see `XDR_RedTeam_Engagement_Plan_2026-06-09.md`.

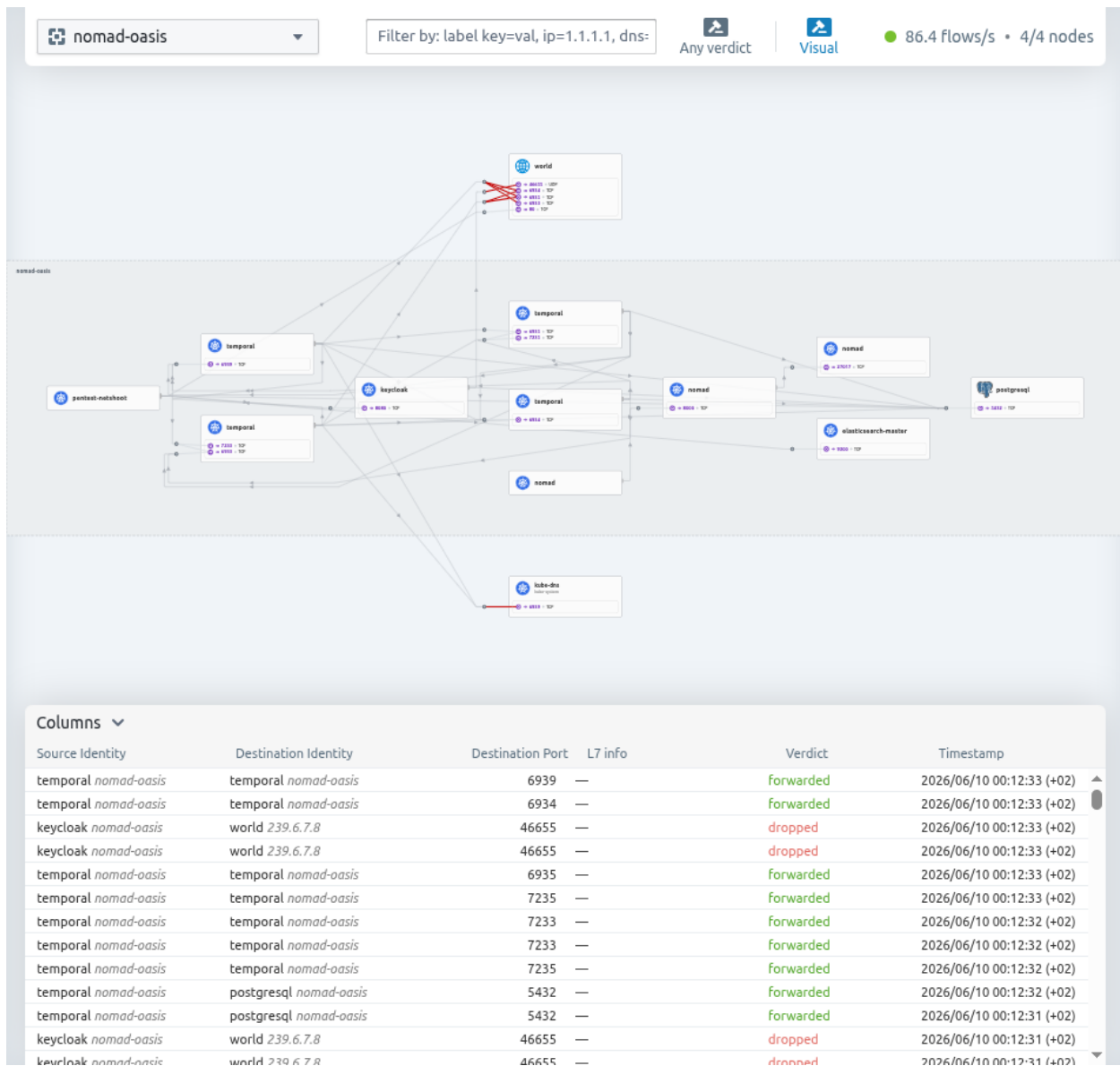


Figure 3: Hubble network map — attacker pod lateral connections

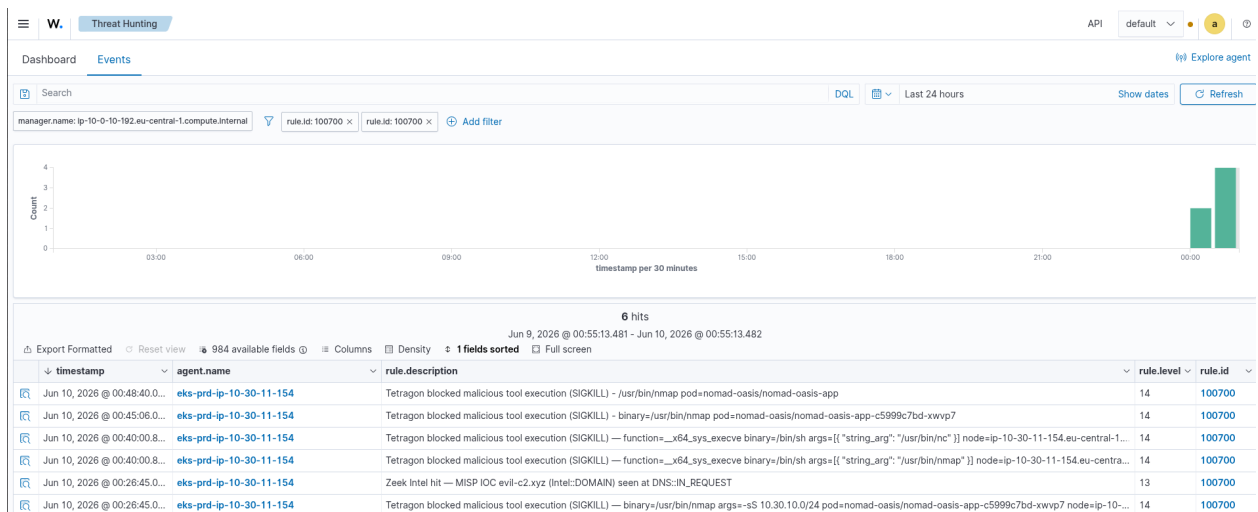


Figure 4: Post-fix: Tetragon SIGKILL of nmap, rule 100700 (level 14)